

Disk Forensics Process Flows

Acquisition → artifact creation → grounded reports

Complete source-traced process diagrams, stage gates and failure paths.

Includes native parsing, recovery states, suspicious images/videos, text-only RAG and PDF/DOCX report generation.

Readiness does not guarantee that inaccessible or overwritten evidence was recovered.

Contents and diagram key

Disk / Evidence-to-report overview	3
The 13 controlled stages	4
01 / Intake and segment preflight	5
02 / Read-only source preparation	6
03 / Extraction, storage and finalization	7
04 / Manifest-to-artifact materialization	8
05 / Native parsing and format dispatch	9
Priority / Web, email and WhatsApp-linked media	10
06 / Recovery and deleted-data evidence states	11
07 / OCR and searchable text	12
08 / Image and video evidence review	13
09 / Normalize, correlate and classify artifacts	14
10 / Text-only RAG chunking	15
11-12 / Entities and evidence graph	16
13 / Validation and readiness	17
The single progressAgent supervisor	18
Report / Intake, scope and evidence selection	19
Report / Grounded section generation	20
Report / Suspicious Activity evidence panels	21
Report / Common section and page structure	22
Report / Shared layout, review and export	23
Verification, performance and implementation map	24

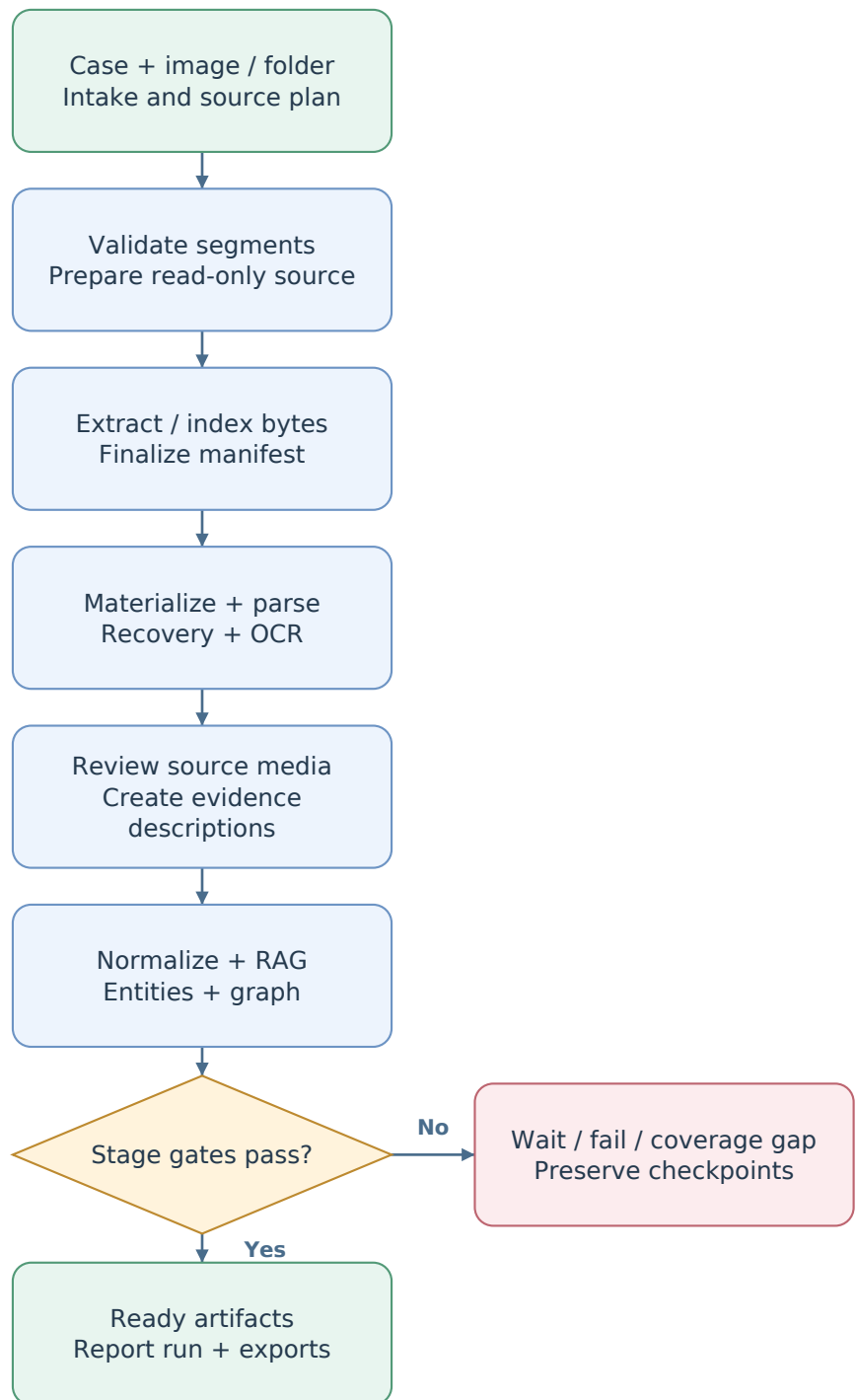
Diagram key

 Process	 Gate / branch	 Retained evidence
 Key / crypto	 Gap / failure	 External workflow

Solid arrows show implemented transitions. Dashed arrows show an explicitly external or conditional path. A gap is evidence of a limitation; it does not establish that the requested data never existed.

Disk / Evidence-to-report overview

Every result traces back to an immutable acquisition or a clearly identified derivative. Extraction, artifact creation, RAG and reporting have separate completion gates.



Resource policy One active serial stage per job; up to four CPU evidence workers inside a stage. OCR and vision share a single GPU permit. Embeddings are not a stage.

Evidence policy Do not infer deleted status, content, identity or suspicious intent from a filename. Record a source locator, hash, evidence state and the reason for each observation.

Implementation: backend/app/services/forensic_serial_pipeline.py; disk.py; report_generator.py

The 13 controlled stages

The same serial controller governs Disk evidence after acquisition handoff. One stage owns the job at a time; bounded workers may operate inside a stage.

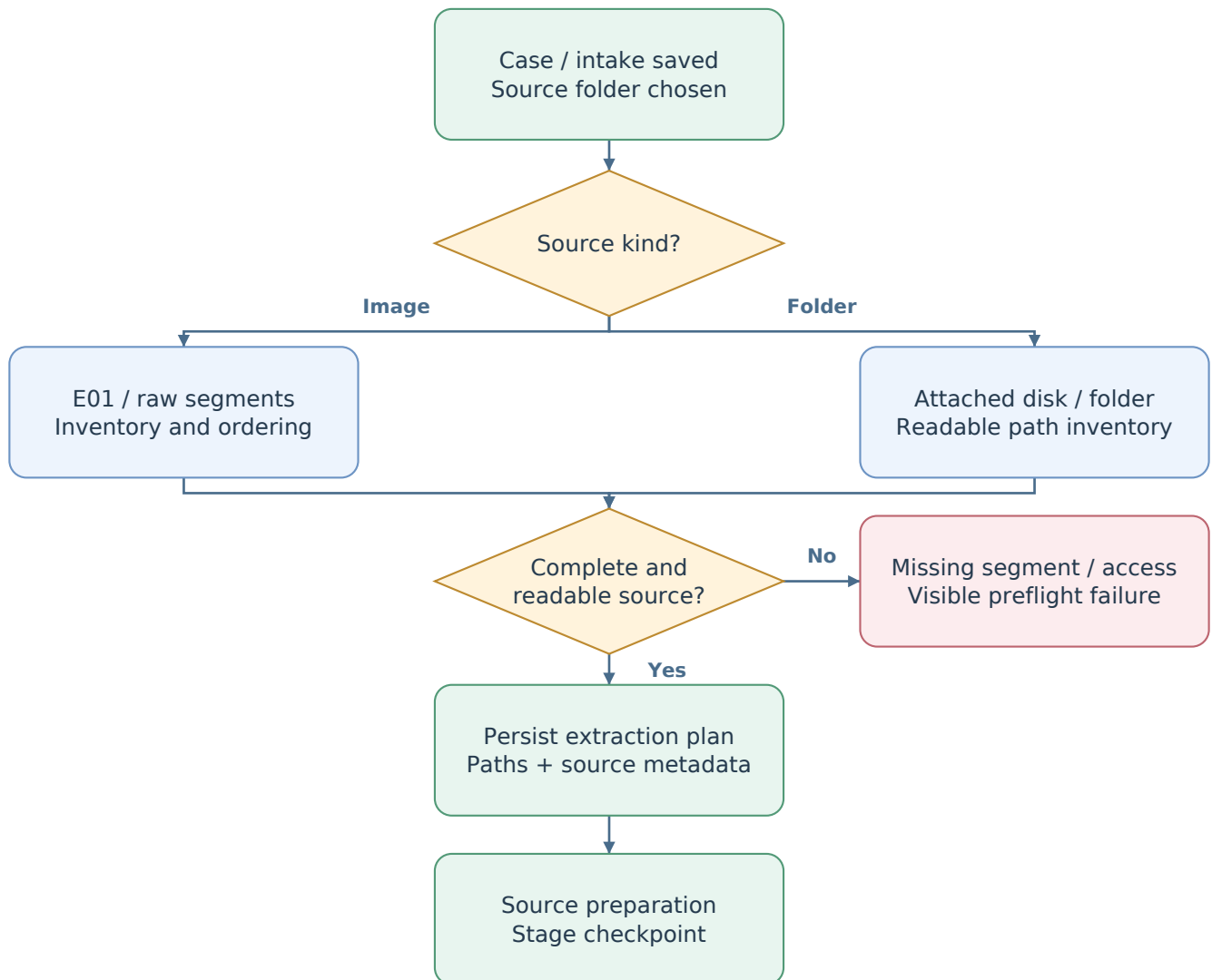
Stage / key	Process agent	Required output
01 intake	segments_agent	Source validation and an extraction plan
02 source_prepare	virtual_disk_agent	Read-only source access / prepared mobile acquisition
03 extraction	extraction_agent	Acquisition completion and a finalized manifest
04 materialize	materialize_agent	Manifest files become durable artifact records
05 parse	parse_agent	Structured records, text and decrypted WhatsApp databases
06 recovery	recovery_agent	Retained deleted bytes and residual candidates
07 ocr	ocr_agent	Text from acquired image/document pixels
08 media_review	media_review_agent	Source images and sampled video frames with descriptions
09 inventory	artifacts_agent	Normalize, classify and correlate evidence
10 chunk	chunk_agent	Text-only RAG: 2500 characters; 800 overlap
11 enrichment	entity_agent	Source-backed entities and enrichment
12 graph	neo4j_agent	Idempotent evidence relationships
13 validation	validation_agent	Stage gates, counts and explicit coverage gaps

State meanings Ready means the configured gates completed. `coverage_complete` and exception counters still describe missing or inaccessible evidence. A completed acquisition does not prove that every possible artifact was captured.

Implementation: `backend/app/services/forensic_serial_pipeline.py`; `forensic_serial_stages.py`; `forensic_serial_policy.py`

01 / Intake and segment preflight

Choose the accessible source folder before processing. Intake links the job to its case and records the intended examination.



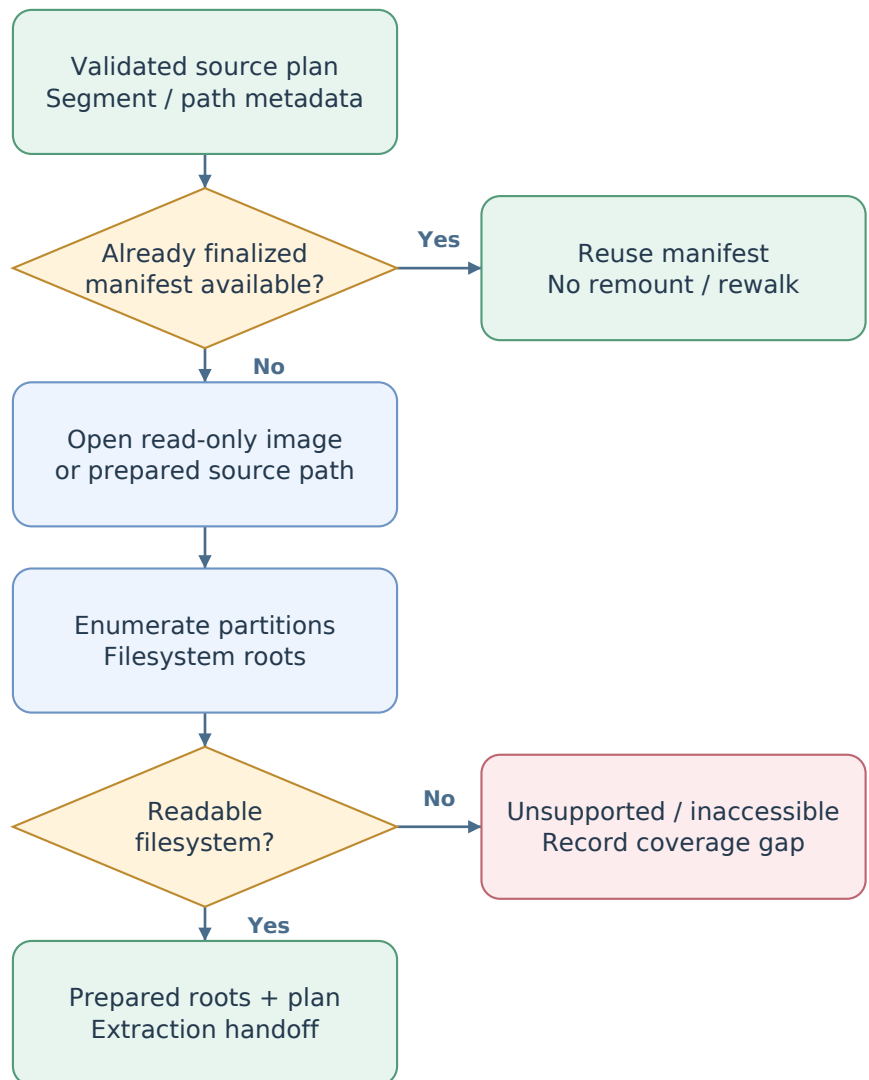
Fail early Incomplete image segments, an unreadable selected directory or an inaccessible drive must be visible before a parser job is dispatched.

Resume Keep the source plan and progress checkpoint. A previously finalized acquisition can hand off to artifacts without repeating the image walk.

Implementation: backend/app/services/forensic_serial_pipeline.py; virtual_disk.py; extracted_disk.py

02 / Read-only source preparation

The preparation step exposes acquired source bytes to extraction. It does not alter the original evidence image.



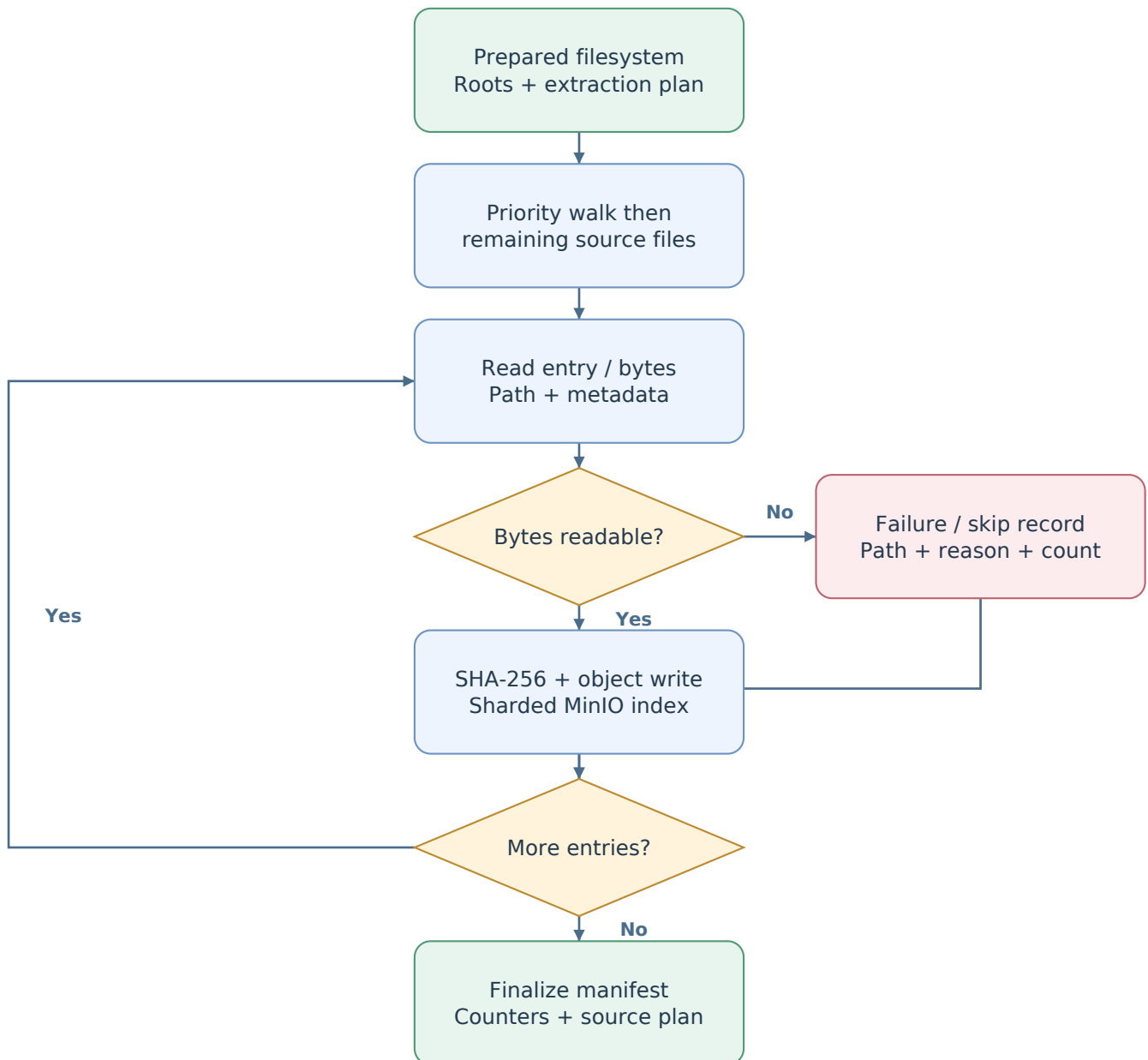
Source integrity Keep original segment/path identifiers. Partitions and filesystem entries remain linked to their source image.

Support boundary The flow records the filesystems and bytes the implemented readers expose. An unsupported encrypted volume requires an authorized decrypted source; preparation is not universal volume decryption.

Implementation: backend/app/services/virtual_disk.py; backend/app/services/disk.py

03 / Extraction, storage and finalization

Extraction walks readable roots, stores acquired bytes and records a finalized index. File-level failures remain explicit.



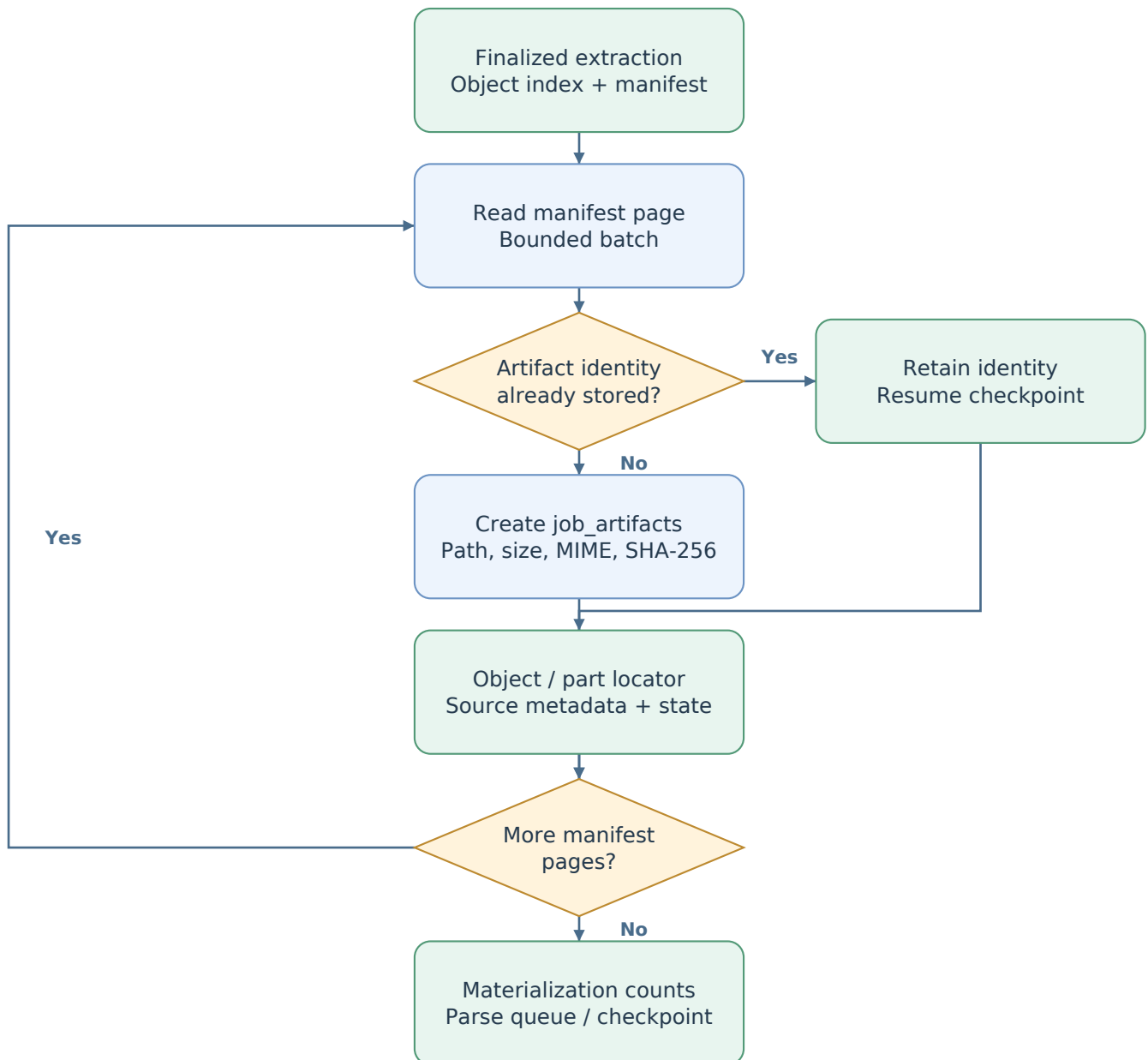
Finalize barrier The manifest is the durable handoff. Acquisition milestones are recorded at finalization; later evidence stages do not silently rewrite the original image.

Performance Use resumable extraction plans, sharded indexes and bounded file workers. Counts include failed and skipped paths so a fast partial acquisition cannot masquerade as complete coverage.

Implementation: backend/app/services/extracted_disk.py; virtual_disk.py; forensic_serial_stages.py

04 / Manifest-to-artifact materialization

Each acquired file receives a stable artifact identity before parsing or model review.



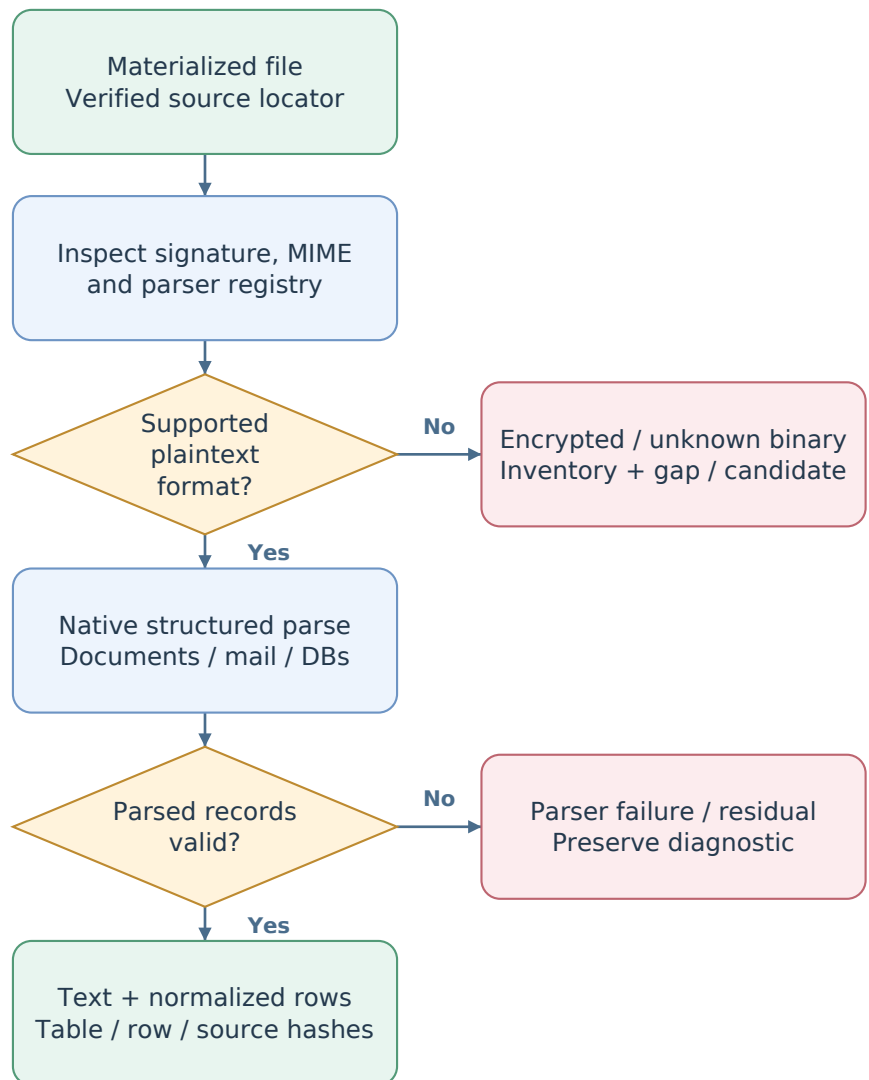
Provenance An artifact can be an acquired file or an identified derivative. Its path, content hash, storage locator and evidence state must distinguish those cases.

Bounded work Paging and stable identities support large acquisitions without duplicate artifact rows after a retry.

Implementation: backend/app/services/forensic_serial_stages.py; forensic_priority_evidence.py

05 / Native parsing and format dispatch

Identify content from signatures and structure as well as extensions. Binary candidates stay labeled when their schema or meaning is unverified.



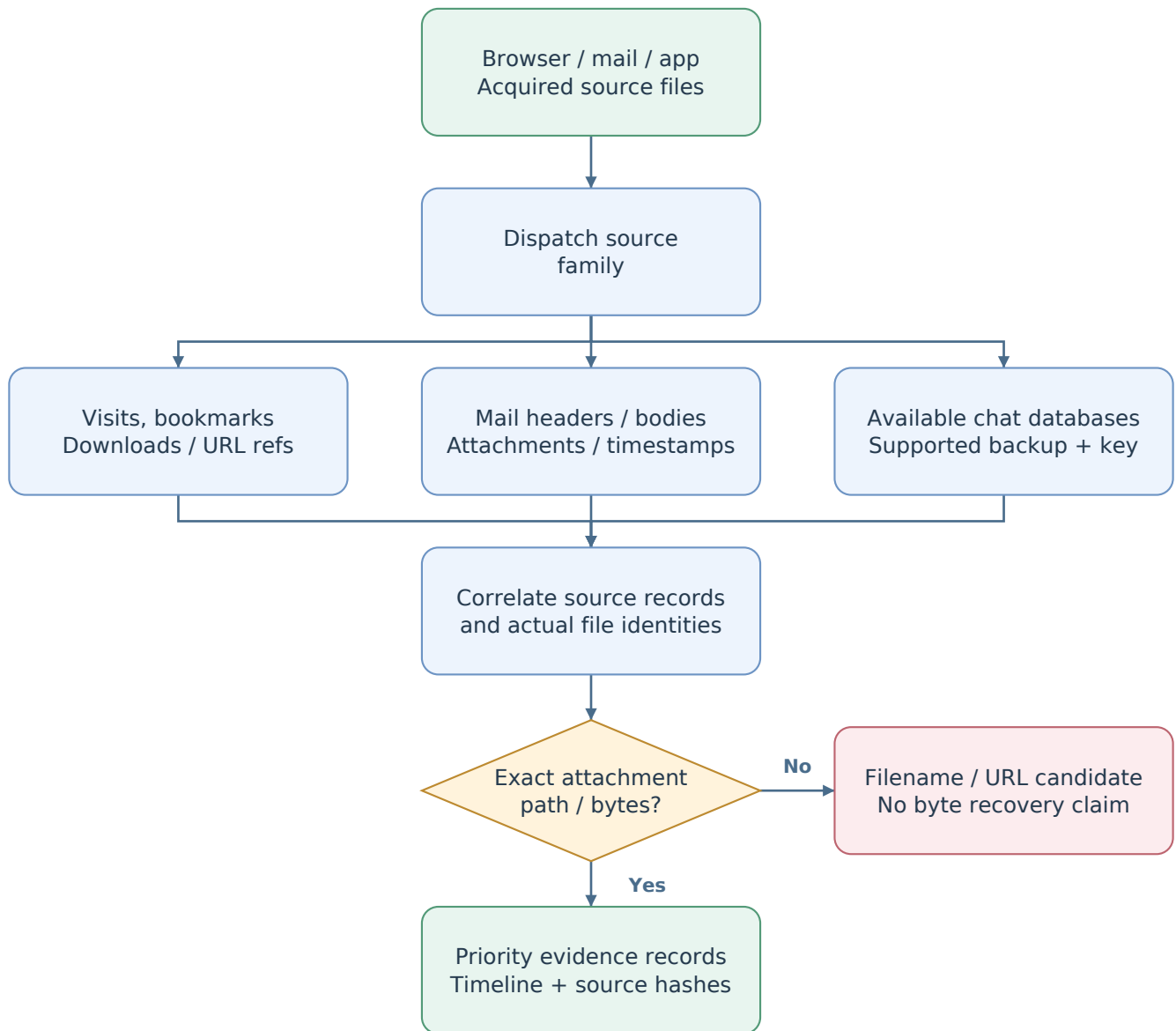
Supported families EML, EMLX, MSG, MBOX/MBX and attachments; office/PDF/text documents; JSON/JSONL/NDJSON, XML, CSV/TSV, plist/bplist and supported SQLite tables.

Quality Native database rows are stronger than unverified binary or protobuf fragments. A filename or printable string is not enough to call a file a conversation database.

Implementation: backend/app/services/forensic_serial_stages.py; forensic_priority_evidence.py; forensic_priority_evidence.py

Priority / Web, email and WhatsApp-linked media

Priority records preserve the distinction between actual browser events, copied URLs and attachment references.



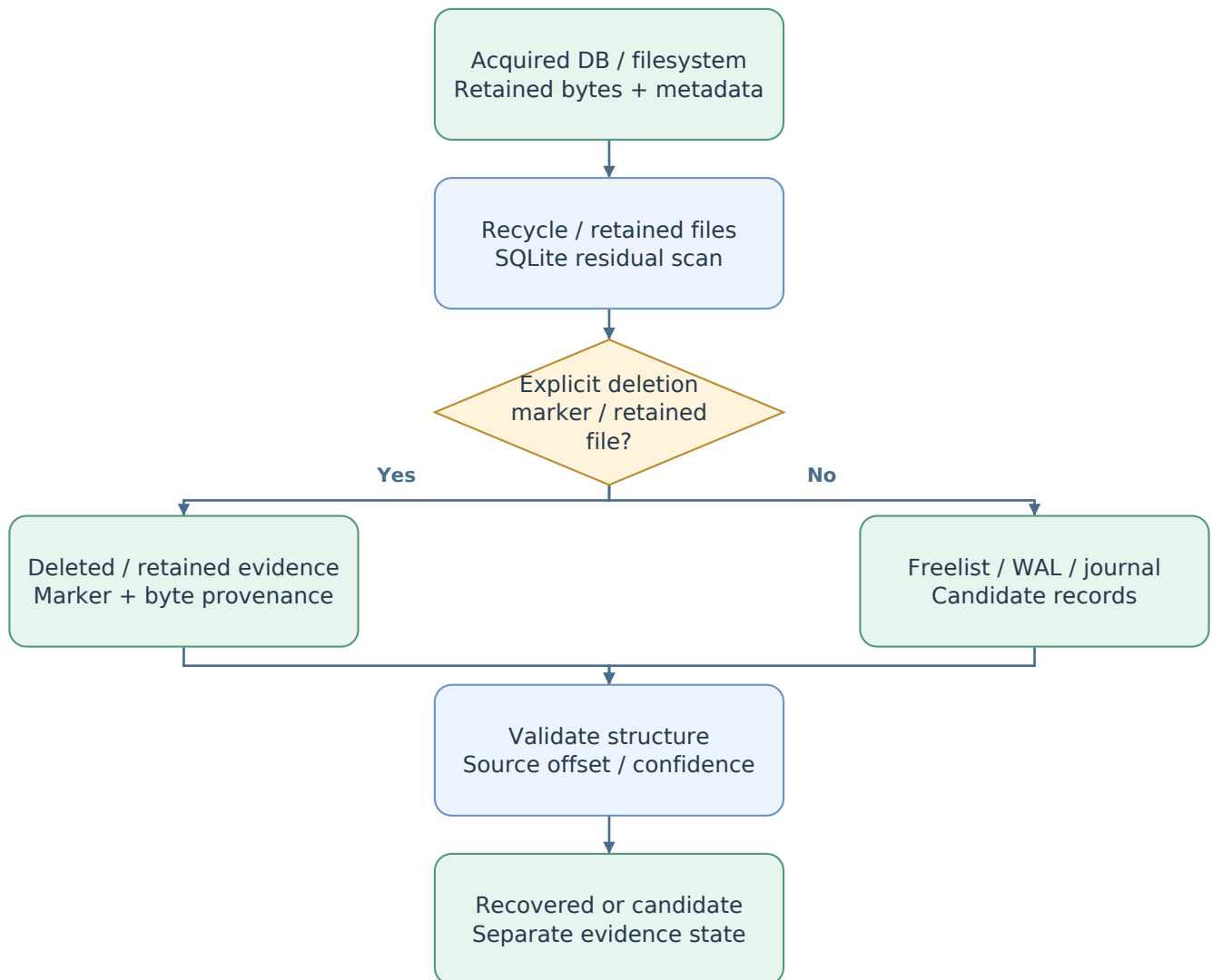
Web coverage Collect acquired history, bookmarks, download records, cookies and login metadata. Protected credentials and encrypted web stores are inventoried unless a supported decryption route is available.

WhatsApp Disk-resident WhatsApp databases/backups use available matching keys and the same authenticated parser path documented in the Mobile PDF. A WhatsApp web URL does not prove decrypted chats were recovered.

Implementation: backend/app/services/forensic_priority_evidence.py; forensic_priority_evidence.py; mobile_forensic/parsers/whatsapp_modern.py

06 / Recovery and deleted-data evidence states

Recovery examines acquired retained bytes. It cannot reconstruct overwritten or unavailable content merely from metadata.



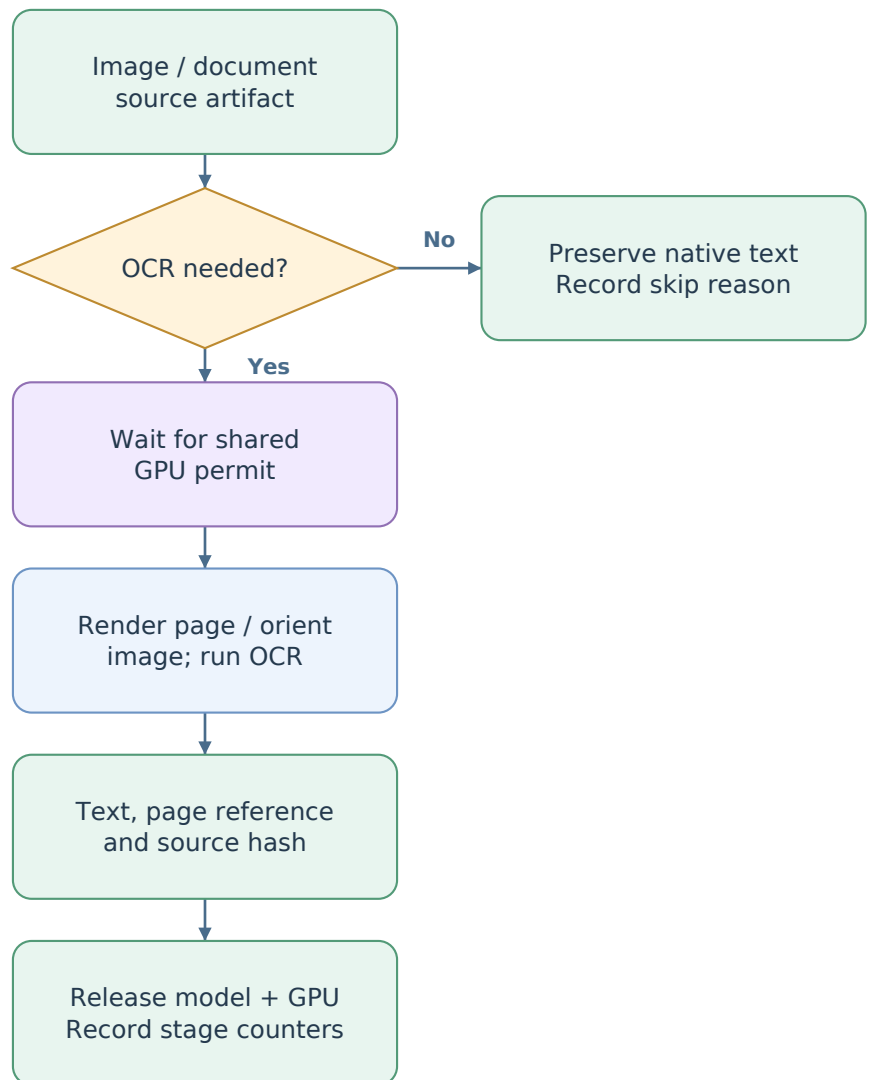
Interpretation A row found in an old backup is historical evidence, not automatic proof that a user deleted the current chat. Residual candidates need corroboration.

Limits This flow does not promise recovery of all unallocated image sectors or encrypted/overwritten deleted files. The source reader and retained bytes determine actual coverage.

Implementation: backend/app/services/forensic_serial_stages.py; mobile_forensic/recovery/analyzers.py; forensic_priority_evidence.py

07 / OCR and searchable text

OCR processes acquired pixels and document pages. It records success, empty results and failures separately.



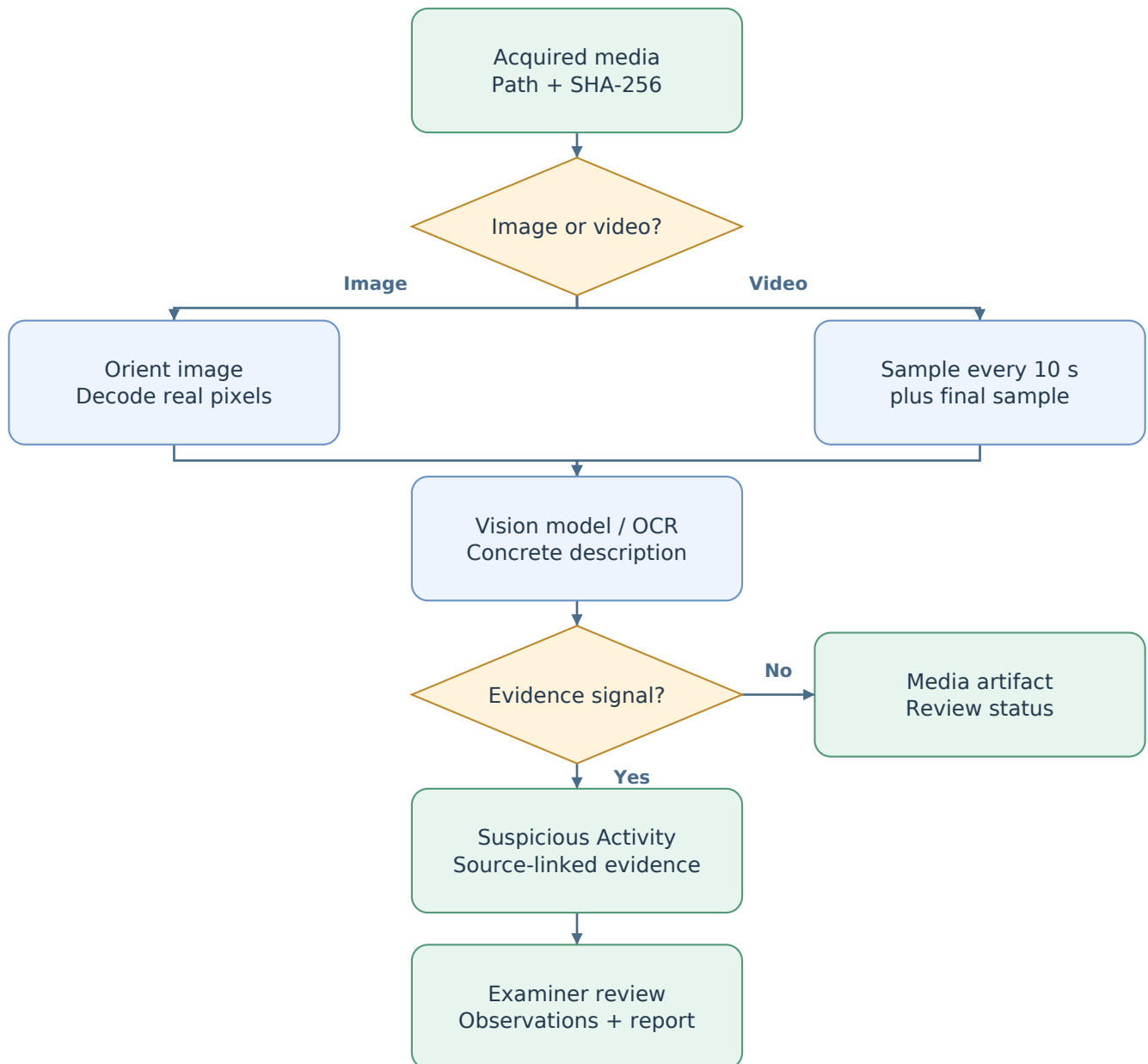
Scheduling OCR and vision share one GPU permit. A legitimate permit or model wait is visible to progressAgent and is not classified as an invalid halt.

Evidence OCR is derived text. Keep the original file, page/frame reference, source hash and OCR status; an OCR guess does not replace the original evidence.

Implementation: backend/app/services/forensic_serial_stages.py; ocr_gpu.py; forensic_media_review.py

08 / Image and video evidence review

Suspicious Activity is created from observable source content and its provenance. Filenames alone do not establish suspicious conduct.



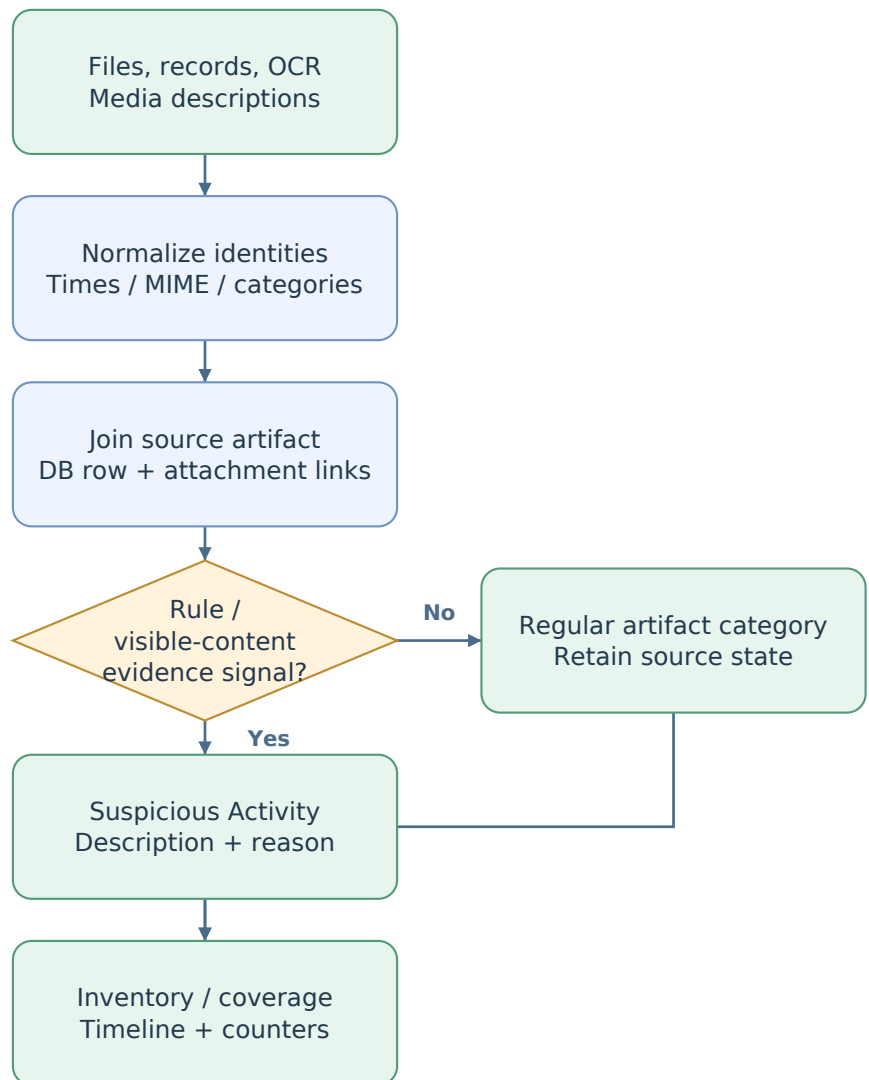
Exact evidence Retain artifact ID, original path/hash, model and description. Video evidence also retains decoded presentation time, derived frame PNG and frame hash.

Scope Images are resized for review up to 1536 px; animated images use the first frame. Video sampling is not an examination of every frame. Missing models or decode failures produce a wait or recorded gap; model interpretation needs examiner review.

Implementation: backend/app/services/forensic_media_review.py; suspicious_activity.py

09 / Normalize, correlate and classify artifacts

Durable source files, native records and derivatives join the inventory without losing their different evidence strengths.



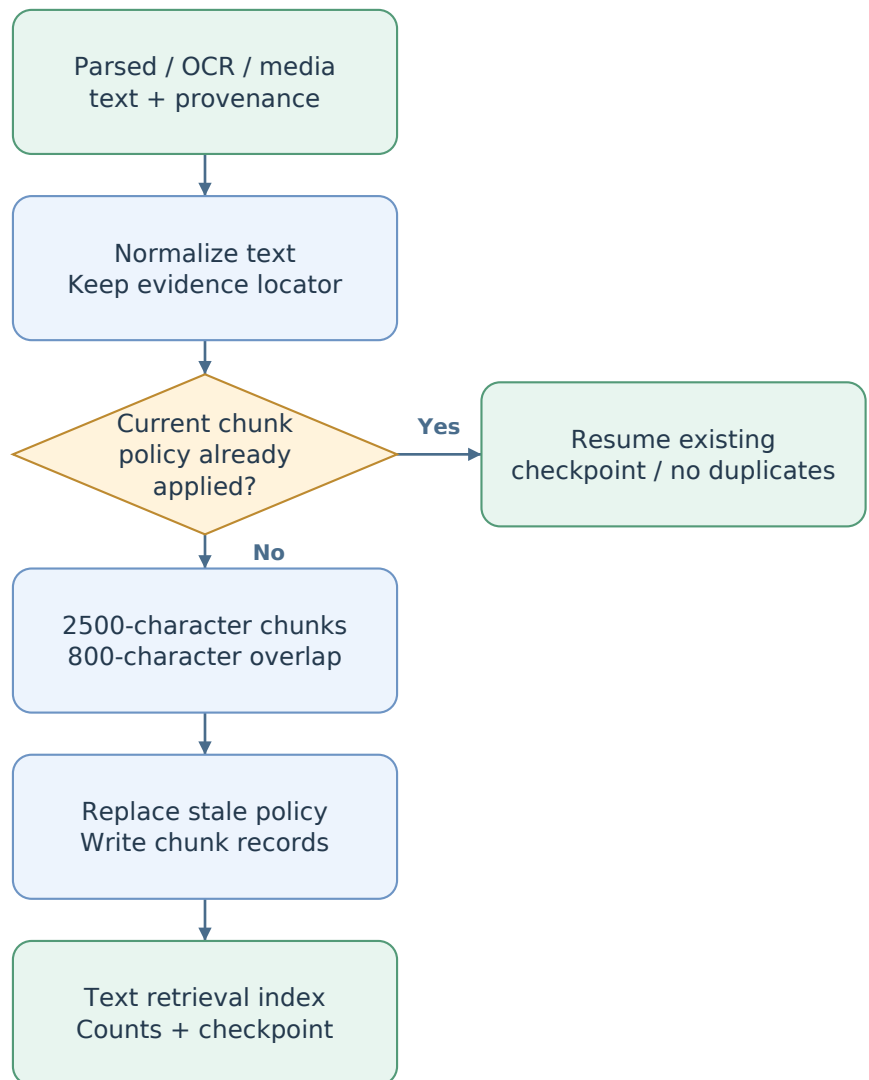
One evidence category Suspicious Activity includes supported chat/mail text, web/download/file/application indicators, OCR and actual image/video observations. Each entry carries a source-backed explanation.

Review Suspicious is a review flag, not a finding of guilt. Unverified fragments and inferred links remain identified as such.

Implementation: backend/app/services/suspicious_activity.py; forensic_priority_evidence.py; forensic_serial_stages.py

10 / Text-only RAG chunking

The embedding stage and forensic vector search are removed. The RAG source is normalized text, parsed content, OCR and evidence descriptions.



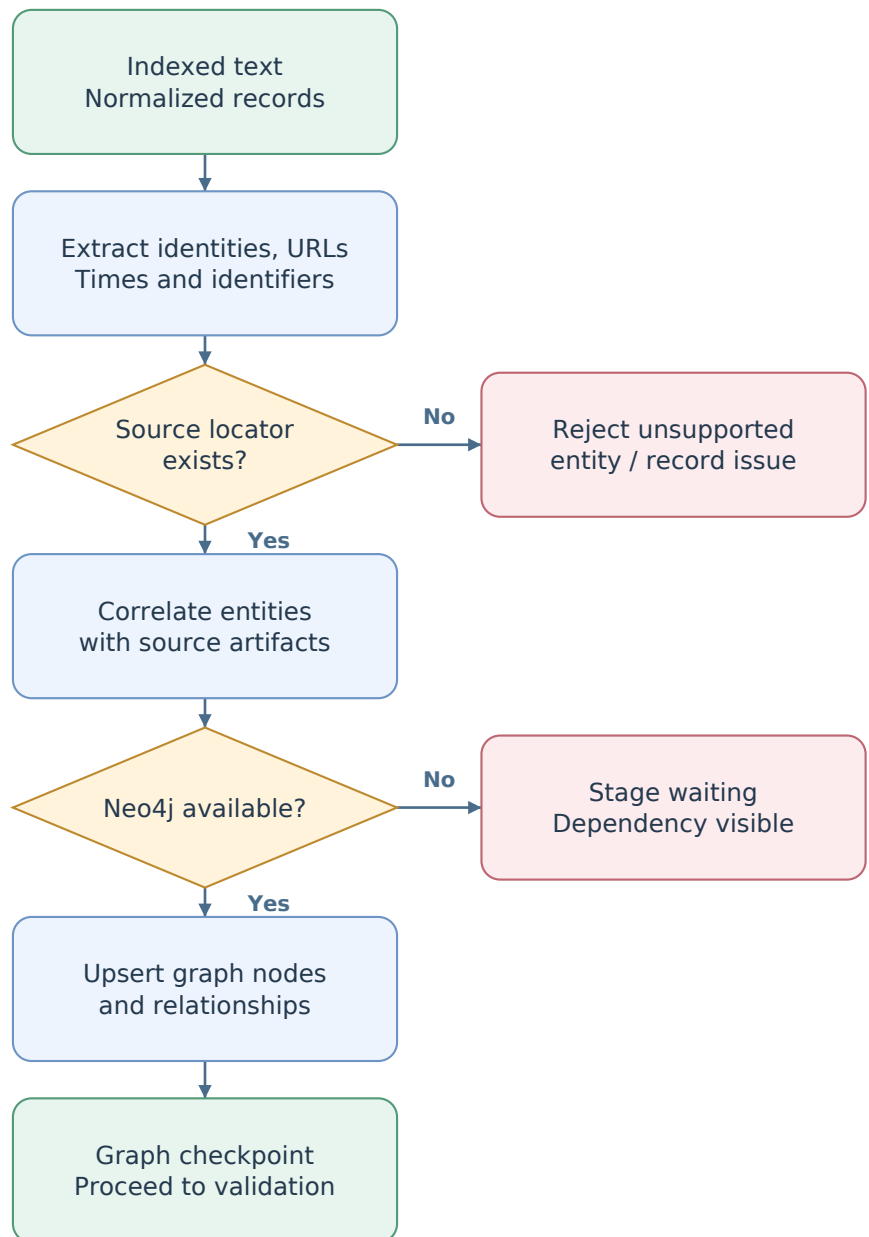
Window definition The limit is characters, not tokens. Headers count inside the 2500-character window. Maximum nominal advance is 1700 characters; boundaries can adjust to the content.

Query / report Retrieval uses text search and evidence references. No GPU embeddings or CLIP retrieval are required. Indexing source descriptions does not add facts beyond the cited source content.

Implementation: backend/app/services/forensic_serial_policy.py; forensic_serial_stages.py; rag_index.py

11-12 / Entities and evidence graph

Enrichment runs after durable chunks. Graph writes preserve source relationships and use idempotent keys.



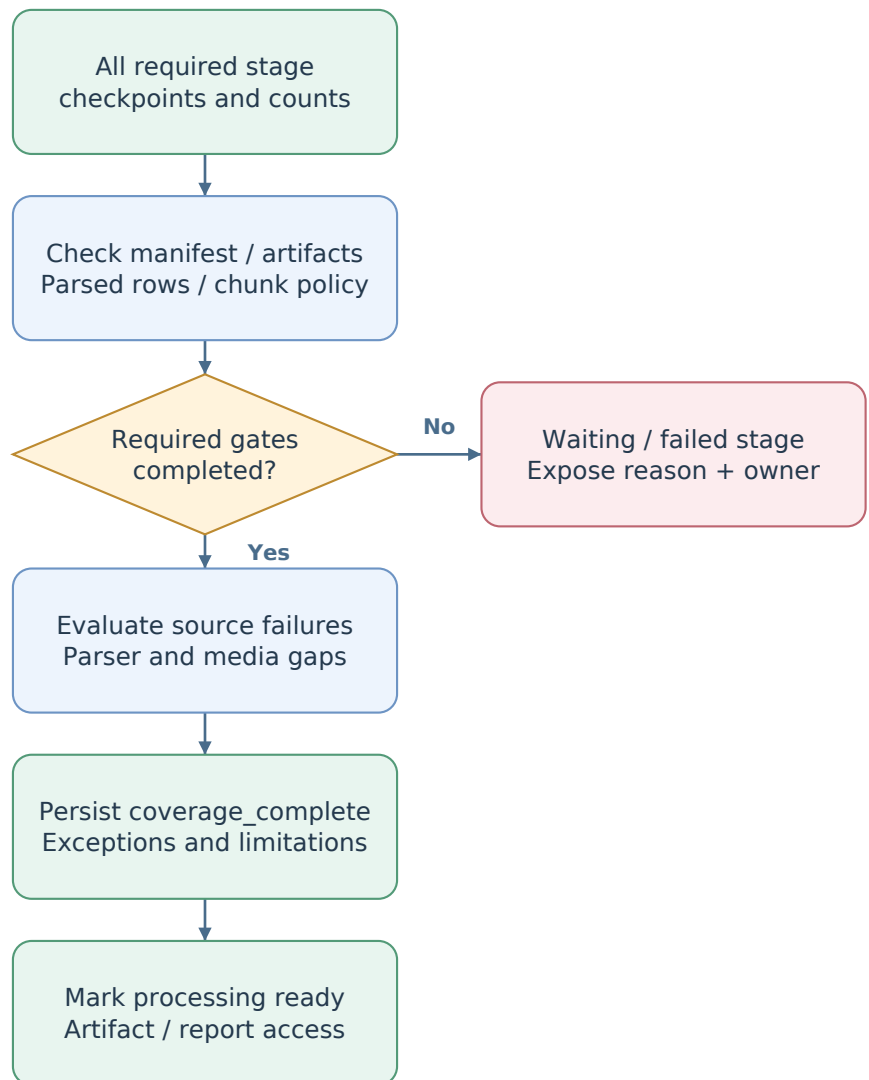
Identity A chat account, sender, image path or URL links to the actual acquired record. Matching names or filenames remain weaker correlation signals.

Dependency gate A graph dependency wait holds the serial gate. `progressAgent` checks the wait reason and retries within policy; it must not report the job ready while the required stage remains waiting.

Implementation: `backend/app/services/forensic_serial_stages.py`; `forensic_serial_pipeline.py`; `forensic_priority_evidence.py`

13 / Validation and readiness

Validation compares required stage outputs and checkpoints. Coverage status is retained alongside readiness.



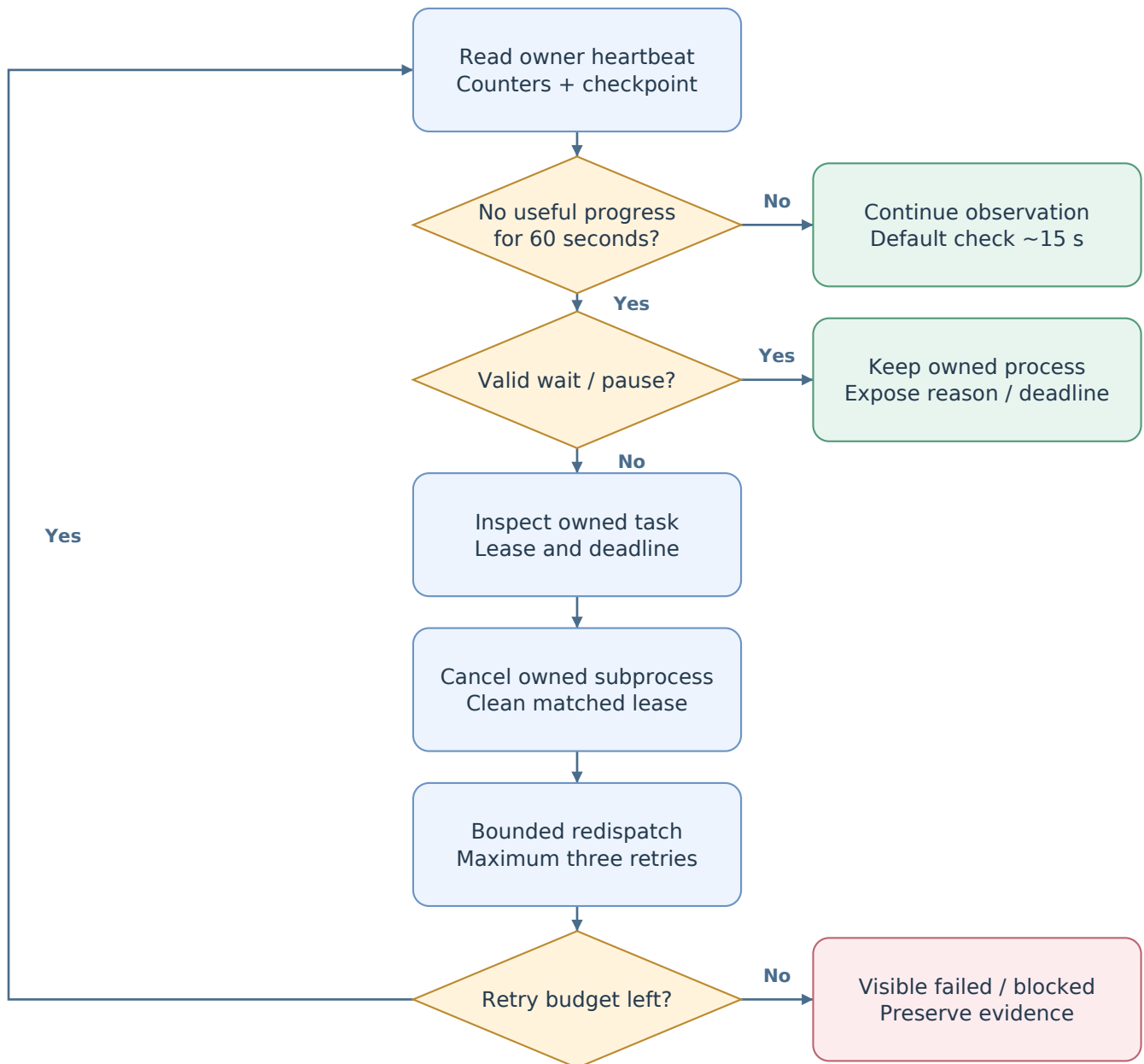
Readiness vs completeness Ready means this acquisition finished its configured stages. An unsupported encrypted source, missing bytes or recorded decode failure remains an explicit coverage limitation.

Retry Resume only incomplete derived stages and preserve acquired bytes. Report regeneration uses new derived records rather than retroactively changing a saved report.

Implementation: backend/app/services/forensic_serial_pipeline.py; forensic_serial_stages.py

The single progressAgent supervisor

progressAgent is the sole outside-process coordinator. Observe, Performance and Repair huddles are retired; queued legacy huddles perform no work.



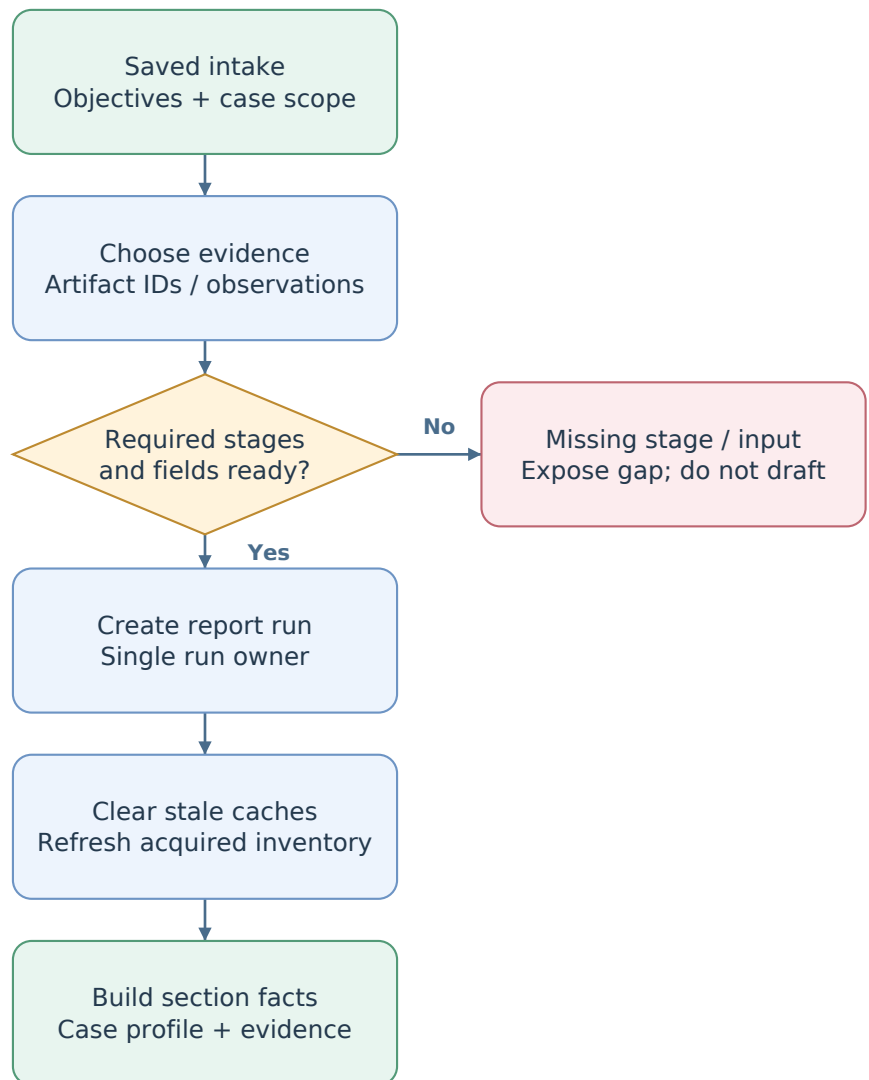
Valid waits User pause, a shared GPU permit, a required model/dependency, or an operation inside its expected deadline may legitimately exceed one minute. A fresh heartbeat alone is not useful progress.

Ownership / cleanup Recovery stops only the owned job process and matched leases. Routine legacy huddle lines are hidden from consoles; historical database rows and processing/evidence logs remain intact.

Implementation: backend/app/services/progress_agent.py; forensic_serial_pipeline.py

Report / Intake, scope and evidence selection

A report run starts from saved case intake and selected evidence. Disk uses image/source details and the finalized extraction.



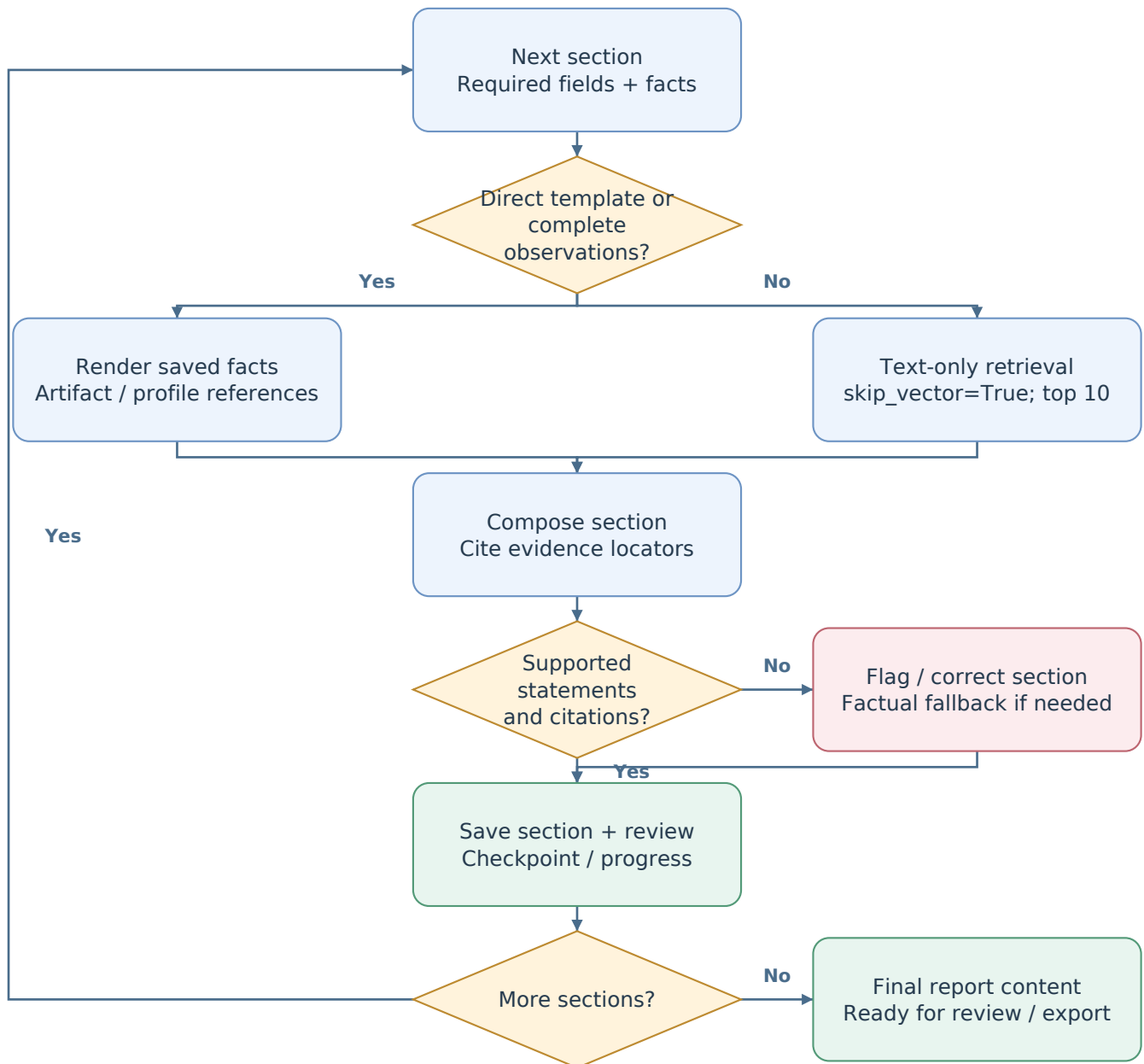
Refresh scope Refresh reads the existing acquisition. Report generation does not acquire a missing phone database or remount an unavailable original source to invent evidence.

Selection Retain selected artifact counts, objectives, procedure and examiner observations. Saved historic reports keep their content; a new run is needed to include new derived evidence.

Implementation: backend/app/routers/report.py; backend/app/services/report_generator.py

Report / Grounded section generation

Sections share a deterministic order. Structured facts and saved observations are preferred where the report has direct evidence.



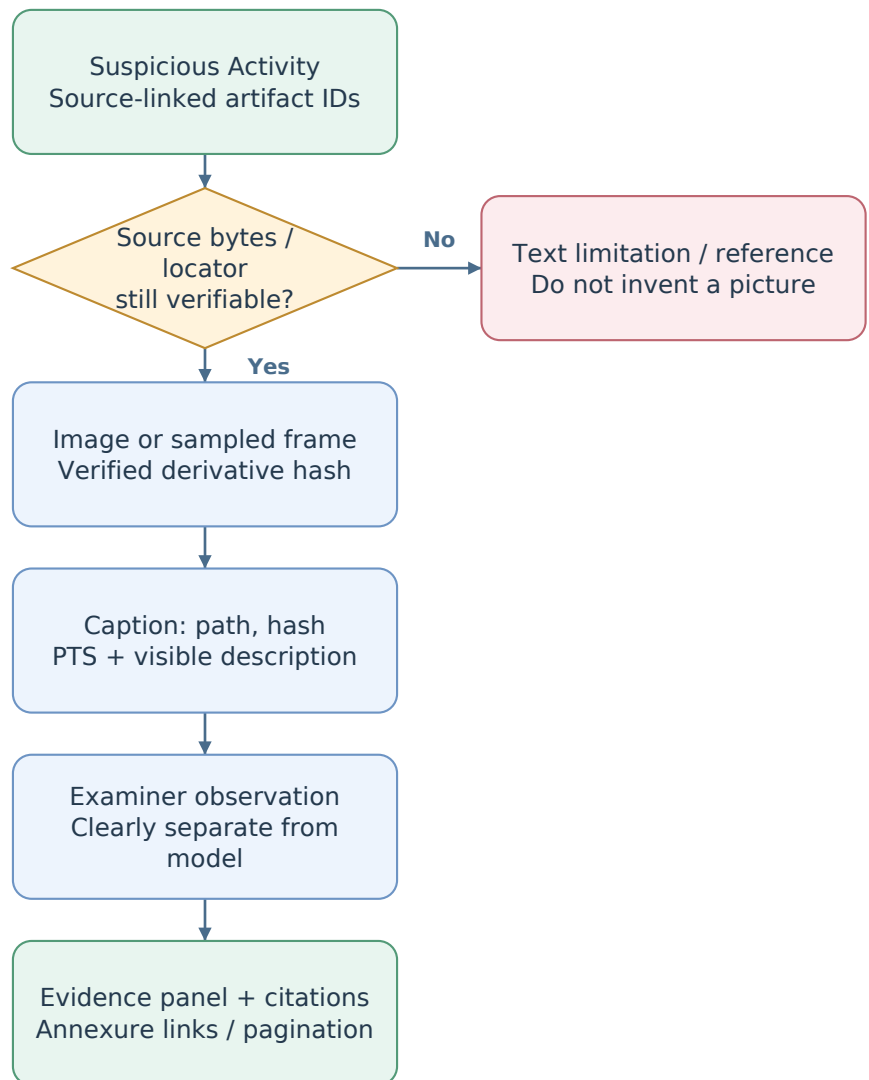
Model boundary The model can describe source facts. Unsupported claims, incomplete evidence and failed generation are reported explicitly; factual templates provide fallback content.

Resuming Section checkpoints and a single report owner support pause/resume. progressAgent tracks actual work, section state and valid model/dependency waits.

Implementation: backend/app/services/report_generator.py; report_generator_agent.py; report_renderer.py

Report / Suspicious Activity evidence panels

The report draws this section from source-linked artifacts and verified media, then allows examiner observations to be added.



Text evidence Chats, email, browsing, downloads and application indicators cite their native record locator and explanation. A URL reference alone does not establish browsing or downloading.

Media evidence Present the actual retained image or sampled video frame with its description. Frame timestamps locate evidence in the video; a sampled frame does not represent the complete video.

Implementation: backend/app/services/suspicious_activity.py; report_renderer.py; forensic_priority_evidence.py

Report / Common section and page structure

The shared renderer keeps the Disk and Mobile report sections aligned. Platform-specific source headings are substituted without changing the surrounding structure.

Report block	Shared content / platform substitution
A / opening	Cover; contents; introduction; scope; tools
A / source	Disk: Forensic Imaging; Mobile: Extraction Summary
A / profile	Disk: OS Information; Mobile: Device Information; User Profile
B	Artifact Summary
C	Objectives, Procedure and Observation
New evidence section	Suspicious Activity
D	Annexure
E	Final Analysis Summary
F	Appendix

Shared formatting A4 page width, section spacing, header, watermark, page numbers, evidence captions and annexure links use the same rendering path.

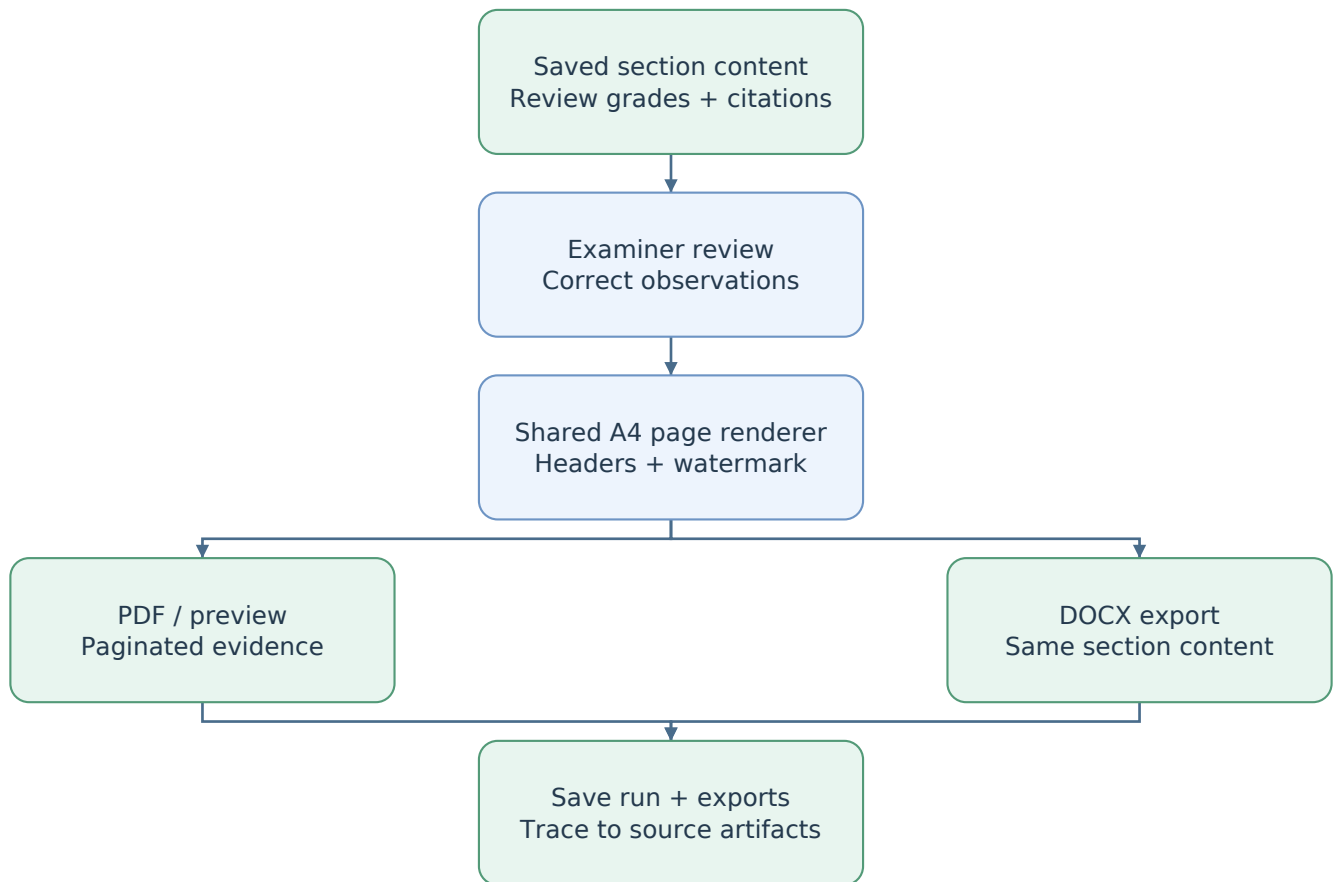
Content and pagination Fourteen logical sections are saved in the report run. Evidence volume determines the page count; identical layout does not imply identical length.

Suspicious Activity This is a dedicated report section and artifact category. It includes source-linked reasons, real retained media or sampled frames, and separate examiner observations.

Implementation: backend/app/services/report_generator.py; report_renderer.py; report_pdf_export.py; report_docx_export.py

Report / Shared layout, review and export

Disk reports use the shared report renderer and section order. The number of pages changes with the evidence; page styling and alignment are shared.



Media alignment Use verified source pictures and derived video frame images with captions, artifact IDs, paths/ hashes and frame timestamps. Keep examiner observations distinct from model descriptions.

Limits The shared layout does not force Disk and Mobile reports to have identical page counts. Missing source pixels or unverified deleted-data candidates must not be presented as exact recovered media.

Implementation: backend/app/services/report_renderer.py; report_pdf_export.py; report_docx_export.py

Verification, performance and implementation map

Source tracing describes what the package does. Live hardware acceptance is a separate result.

Area	Current contract / main implementation
Acquisition	disk.py; virtual_disk.py; extracted_disk.py — read-only access, manifest finalization, resumable walk
Serial / resources	forensic_serial_pipeline.py / policy.py — 13 stage gates, four CPU workers, one shared OCR/vision GPU permit
Parsing / evidence	forensic_priority_evidence.py / evidence.py; mobile parsers — native records and explicit evidence states
Media / suspicious	forensic_media_review.py; suspicious_activity.py — real pixels and frame provenance
Search / report	Text chunks 2500 / overlap 800; report_generator.py; shared renderer and PDF/DOCX exporters
Supervisor	progress_agent.py — useful-progress checks after 60 s; matched-owner bounded recovery
Live acceptance	Test-Live-Phone-GPU.ps1 and LIVE-PHONE-GPU-VERIFICATION.md — real phone, actual bytes and GPU inference required
This release	Disk/Mobile pipeline code retained from the latest supplied project. Laptop/central scanner fixes and these flow documents are added separately.

Performance already in code Bounded workers, manifest paging/checkpoints, source-preserving resume, text-only RAG and serialized GPU inference reduce contention. A waiting model or large file is evaluated against its stage deadline.

Deployment Use the existing Disk/Mobile serial deployment instructions in DISK-MOBILE-SERIAL-RELEASE.md. The scanner update scripts affect the vulnerability services, not the source evidence acquisition.

Implementation: All service file names above are under backend/app/services unless stated otherwise.